

Yes or No — and How Sure?

Linear models for classification — logistic regression, the sigmoid, the cross-entropy cost, gradient descent, regularisation, and the full toolkit for judging a classifier: confusion matrix, precision, recall, F-score, ROC and AUC.

What is inside

Finance asks Meera a yes/no question, and the straight line of Book 3 breaks. This book builds logistic regression from that failure — the sigmoid, the odds, the cost that stays a bowl, and the same gradient descent as before — then spends equal time on the harder question: how do you know a classifier is any good? Every slide example (the CGPA/IQ job data, the sentiment vector, the skewed confusion matrices, the ROC tables) is worked in full.

1	Why a Straight Line Cannot Say Yes or No	Classification recap, discriminant functions, and the two ways least squares fails on a 0/1 target.
2	The Sigmoid, Odds, and the Decision Boundary	The logistic function and its properties, log-odds, how to read a coefficient, and boundaries straight and curved.
3	The Cost That Stays a Bowl	Why squared error fails, the — log cost, cross-entropy, and the gradient that turns out identical to linear regression's.
4	Fitting by Hand: The Job-Offer Example	One full iteration of gradient descent on the slide's CGPA/IQ data, the sentiment vector, and regularisation with C .
5	Is the Classifier Any Good? The Confusion Matrix	Accuracy and why it lies, precision, recall, specificity, F-score — with the skewed-class tables worked out.
6	ROC Curves and AUC	One classifier, every threshold: how to build an ROC curve from scores, compute the area, and compare models.
7	Rare Classes and Many Classes	What to do when the class you care about is rare, and how a two-class model handles three or more classes.
8	Formula Sheet, Self-Check, and What Comes Next	Everything from Sessions 5–6 on one page, twelve questions with answers, and the bridge to decision trees.

How to read this book

Every chapter follows the same rhythm: a short **story** to set the scene, the **big idea** in one line, an everyday **analogy**, then the **maths step by step** — always the plain version first, and the matrix version only after the plain one is clear. Boxes to watch for:

STORY Meera at Sahi Deal — the running example.	BIG IDEA The one sentence to remember.	THINK OF IT LIKE A real-life picture of the concept.
STOP AND THINK A question to try before reading on.	OUR ANSWER The answer, right after the question.	CAREFUL A common mistake and how to avoid it.
MATHS, STEP BY STEP Every line of algebra, nothing skipped.	WORKED EXAMPLE Real numbers, worked in full.	EXAM TIP How this topic is asked, and how to score.

IN ONE BREATH

The chapter in a few lines.

CHAPTER 1

Why a Straight Line Cannot Say Yes or No

Classification recap, discriminant functions, and the two ways least squares fails on a 0/1 target.

STORY

Sahi Deal now offers car loans. Priya from Finance brings Meera a table: applicant's CGPA-style credit score, an "IQ"-style stability score, and whether they repaid. "Predict yes or no for new applicants." Meera, fresh from Book 3, fits a straight line to the 0/1 column and draws a cut-off at 0.5. It works — until Priya adds a few very safe applicants and the cut-off drifts, and Meera notices the line is predicting 1.3 for someone. "A probability of 130%?"

Classification, briefly

In classification the output is **discrete**: $f(\mathbf{x}) \in \{\text{yes, no, maybe}\}$, or in the binary case $f(\mathbf{x}) \in \{0, 1\}$. The objective is to **minimise the misclassification rate** — or, as we will see, something more careful than that. Two families of classifier:

Probabilistic generative

Model how each class *generates* data — the joint distribution $P(\mathbf{X}, Y)$ — then use Bayes' rule to classify. (Naïve Bayes, Module 8.)

Probabilistic discriminative

Model the *boundary* between the classes directly — $P(Y | \mathbf{X})$. Logistic regression, this book's subject, is the classic example.

The linear discriminant

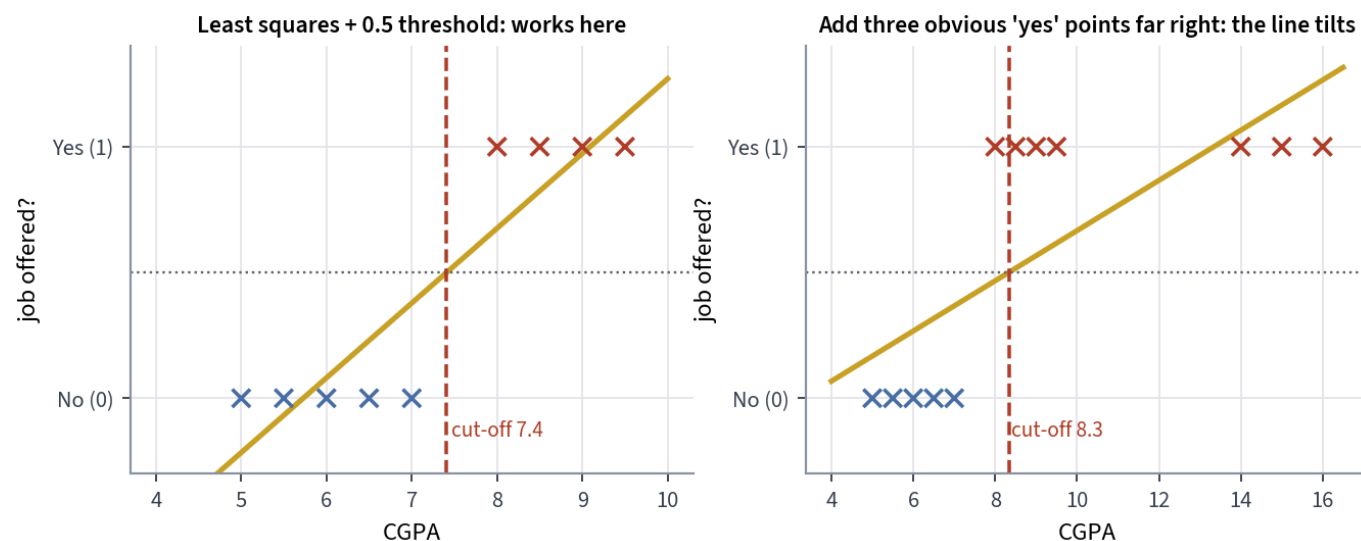
A **discriminant** is a function that takes an input $\mathbf{x}$ and assigns it to one of K classes. The simplest is linear:

$$f(\mathbf{x}, \mathbf{w}) = w_0 + \mathbf{w}^T \mathbf{x}, \quad \text{decision rule: } y = \text{sign}(w_0 + \mathbf{w}^T \mathbf{x}).$$

Predict class 1 when $f > 0$ and class 0 when $f < 0$. The set where $w_0 + \mathbf{w}^T \mathbf{x} = 0$ is a straight line in 2-D, a plane in 3-D, a **hyperplane** in general — the **decision boundary**. Everything in this book keeps this linear boundary; what changes is how we *learn* $\mathbf{w}$ and what we do with f .

First attempt: least squares plus a threshold

Code $y = 1$ for "job offered" and $y = 0$ for "not offered", fit $h_{\theta}(x) = \theta_0 + \theta_1 \text{CGPA}$ by least squares, and predict 1 whenever $h_{\theta}(x) \geq 0.5$. On a tidy dataset this actually works.



Left: the line crosses 0.5 between the two groups; the threshold classifies everything correctly. Right: add three applicants with very high CGPA — obviously "yes", no new information — and the line tilts, the cut-off moves right, and a previously correct "yes" at CGPA 8.0 is now called "no".

TWO FAILURES OF LEAST SQUARES FOR CLASSIFICATION

- 1. Sensitivity to easy points.** Squared error punishes a prediction of 1.8 for a "yes" example as if it were a mistake, so the line bends to accommodate obvious cases and moves the boundary. Bishop's picture on the slide: adding points at the bottom-left drags the least-squares boundary badly, while logistic regression's boundary barely moves.
- 2. Outputs outside $[0, 1]$.** A line is unbounded: $h_{\theta}(x)$ can be 1.3 or -0.4 . Those numbers cannot be probabilities, and "how sure" is exactly what Finance wants to know.

THINK OF IT LIKE

Measuring "will it rain?" with a ruler. A ruler can read 130 cm; rain either happens or it does not. What you want is a dial that is pinned between "certainly not" and "certainly yes" and reads "70% likely" in between. The next chapter builds that dial.

BIG IDEA — WHAT WE NEED

A model whose output is always between 0 and 1, so it can be read as $P(y = 1 | \mathbf{x})$; whose decision boundary is still linear (so it stays simple and learnable); and whose cost function does not over-react to easy examples. Logistic regression is exactly this.

IN ONE BREATH

Classification wants a discrete answer and, ideally, a probability. A least-squares line gives neither reliably: it tilts toward easy points and it leaks outside $[0, 1]$. Keep the linear boundary $w_0 + \mathbf{w}^T \mathbf{x} = 0$; replace the line's output with a squashed, probabilistic one.

CHAPTER 2

The Sigmoid, Odds, and the Decision Boundary

The logistic function and its properties, log-odds, how to read a coefficient, and boundaries straight and curved.

STORY

Arjun draws an S on the whiteboard. "Feed your line into this. Whatever the line says — minus a thousand, plus a thousand — the S turns it into a number between 0 and 1. And the S is not arbitrary: it is the shape you get when you write down what odds are."

The model

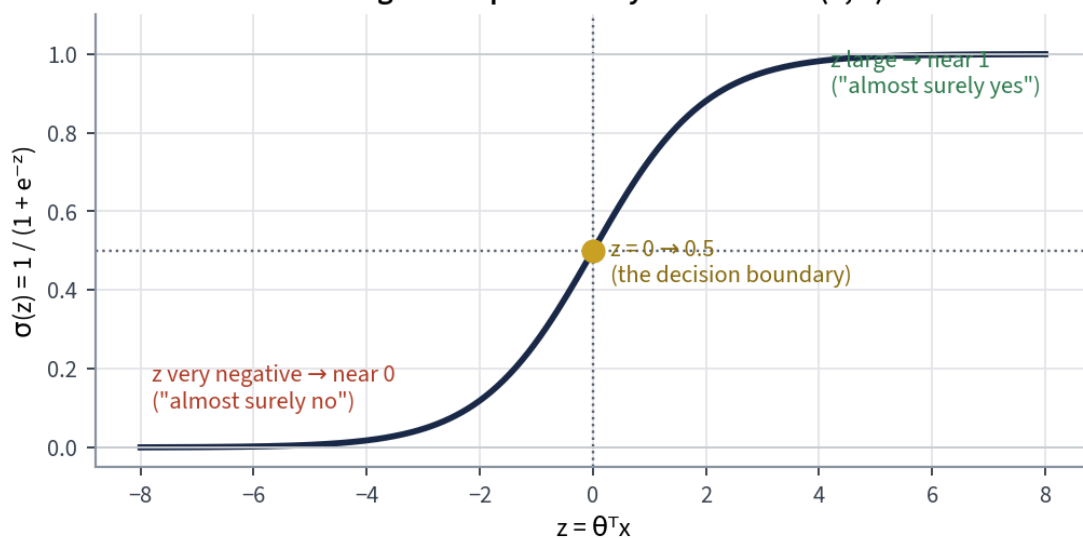
BIG IDEA — LOGISTIC REGRESSION

Compute the same linear score as before, then squash it with the **sigmoid** (logistic) function:

$$z = \boldsymbol{\theta}^T \mathbf{x} = \theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d, \quad h_{\boldsymbol{\theta}}(\mathbf{x}) = g(z) = \frac{1}{1 + e^{-z}}.$$

Read $h_{\boldsymbol{\theta}}(\mathbf{x})$ as the **estimated probability that** $y = 1$ given $\mathbf{x}$: $h_{\boldsymbol{\theta}}(\mathbf{x}) = P(y = 1 \mid \mathbf{x}; \boldsymbol{\theta})$. Predict $y = 1$ if $h_{\boldsymbol{\theta}}(\mathbf{x}) \geq 0.5$, else $y = 0$.

The sigmoid squashes any number into (0, 1)



The sigmoid $g(z)$. It is 0.5 exactly at $z = 0$, climbs steeply near the middle, and flattens toward 0 and 1 at the ends.

"Logistic" is the name of the S-shaped function; "regression" because it still combines a weight vector with the inputs — but the output is a class. A tidy way to picture it (the slide's neuron diagram): inputs $x_1, \dots, x_m$ with $x_0 = 1$ are weighted and summed to $z = \sum_i w_i x_i$; the sigmoid turns z into $\sigma(z)$; a threshold at 0.5 turns that into the predicted label. This is a single artificial neuron.

MATHS, STEP BY STEP — FOUR PROPERTIES OF THE SIGMOID YOU WILL USE

- 1 **Range.** $e^{-z} > 0$ always, so $1 + e^{-z} > 1$ and $0 < g(z) < 1$. As $z \rightarrow +\infty$, $e^{-z} \rightarrow 0$ and $g \rightarrow 1$; as $z \rightarrow -\infty$, $e^{-z} \rightarrow \infty$ and $g \rightarrow 0$.
- 2 **Midpoint.** $g(0) = \frac{1}{1+1} = 0.5$. Hence $g(z) \geq 0.5 \iff z \geq 0$: the model predicts 1 exactly when $\theta^\top \mathbf{x} \geq 0$. You never need to compute the sigmoid to classify — just the sign of z .
- 3 **Symmetry.** $1 - g(z) = \frac{1 + e^{-z} - 1}{1 + e^{-z}} = \frac{e^{-z}}{1 + e^{-z}} = \frac{1}{e^z + 1} = g(-z)$. So $P(y = 0 | \mathbf{x}) = 1 - h_\theta(\mathbf{x}) = g(-z)$.
- 4 **Derivative.** Write $g = (1 + e^{-z})^{-1}$. Then $g' = -(1 + e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = g(z)(1 - g(z))$. This tiny identity is what makes Chapter 3's gradient so clean.

Odds and log-odds: where the S comes from

If p is the probability of "yes", the **odds** are $\frac{p}{1-p}$ — "3 to 1" means $p = 0.75$. Odds run from 0 to ∞ ; their logarithm, the **log-odds** or **logit**, runs over the whole real line — exactly the range of a linear function. Logistic regression is the statement

$$\ln \frac{p}{1-p} = \theta_0 + \sum_i \theta_i x_i = z.$$

MATHS, STEP BY STEP — SOLVING THE LOGIT FOR P

- 1 $\ln \frac{p}{1-p} = z \Rightarrow \frac{p}{1-p} = e^z$.
- 2 $p = e^z(1-p) = e^z - e^z p \Rightarrow p + e^z p = e^z \Rightarrow p(1 + e^z) = e^z$.
- 3 $p = \frac{e^z}{1 + e^z}$. Divide top and bottom by e^z : $p = \frac{1}{e^{-z} + 1} = \frac{1}{1 + e^{-z}} = g(z)$. ✓

So the sigmoid is not a design choice pulled from the air: it is the inverse of the logit. "Linear model for the log-odds" and "sigmoid of a linear model" are the same sentence.

Reading a coefficient: the odds ratio

Because $\ln(\text{odds}) = z$ is linear, increasing x_i by one unit adds θ_i to the log-odds — i.e. **multiplies the odds** by e^{θ_i} . That factor is the **odds ratio**.

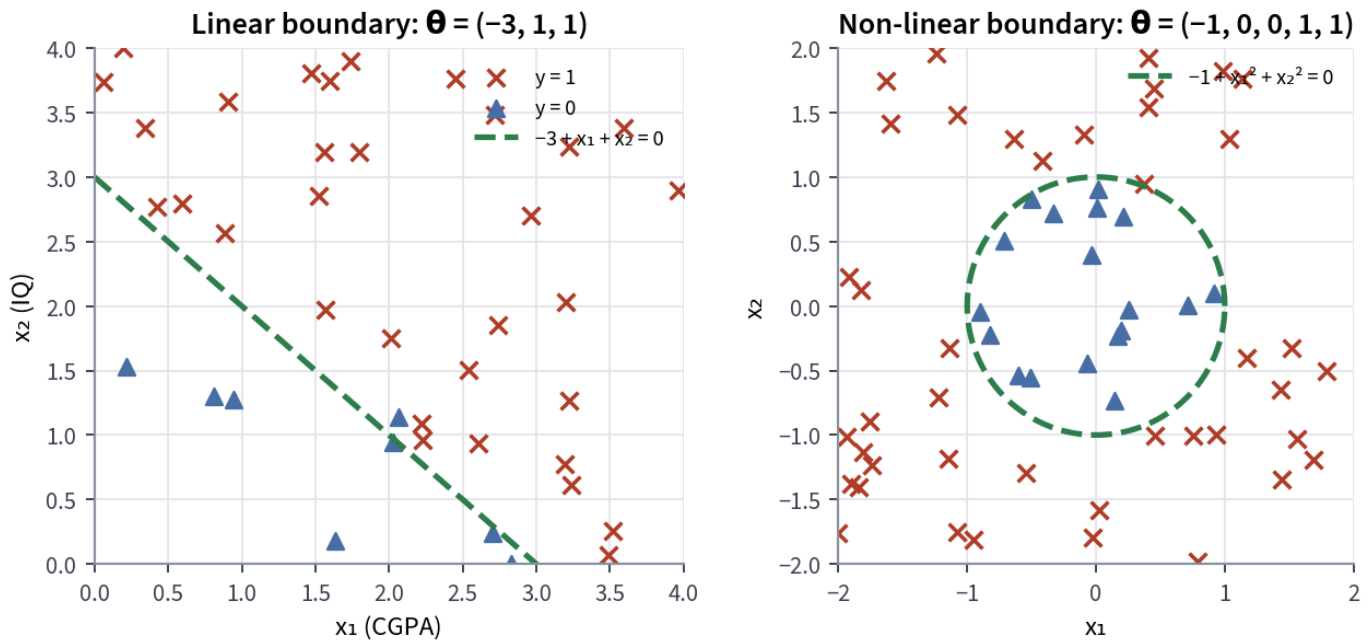
WORKED EXAMPLE (SLIDE)

Model: $z = 0.4 + 0.3 \text{ CGPA} - 0.45 \text{ IQ}$. The CGPA coefficient is 0.3, so $e^{0.3} = 1.35$: the odds of a job offer increase by a factor of 1.35 for every one-unit increase in CGPA, holding IQ fixed. (In statsmodels: `np.exp(model.params)`.) For IQ, $e^{-0.45} = 0.64$ — each extra unit multiplies the odds by 0.64, i.e. lowers them.

Prediction for a candidate with CGPA = 5, IQ = 6: $z = 0.4 + 0.3(5) - 0.45(6) = 0.4 + 1.5 - 2.7 = -0.8$. $h = \frac{1}{1 + e^{0.8}} = \frac{1}{1 + 2.2255} = 0.31$. Since $0.31 < 0.5$ (equivalently $z < 0$): **predicted $y = 0$, no offer**, with about 31% probability of an offer.

The decision boundary

At the boundary $h_\theta = 0.5$, i.e. $z = 0$. The boundary is a property of the **parameters**, not of the data.



Left: $\theta = (-3, 1, 1)$ gives the line $x_1 + x_2 = 3$. Right: with squared features, $\theta = (-1, 0, 0, 1, 1)$ gives the circle $x_1^2 + x_2^2 = 1$.

WORKED EXAMPLES (SLIDES)

Linear. $h_\theta(\mathbf{x}) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$ with $\theta_0 = -3, \theta_1 = 1, \theta_2 = 1$. Predict $y = 1$ if $-3 + x_1 + x_2 \geq 0$, i.e. $x_1 + x_2 \geq 3$. The boundary is the line $x_1 + x_2 = 3$; points above-right are class 1.

Non-linear. Add the features x_1^2 and x_2^2 : $h_\theta(\mathbf{x}) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2)$ with $\theta = (-1, 0, 0, 1, 1)$. Predict $y = 1$ if $-1 + x_1^2 + x_2^2 \geq 0$: outside the unit circle. With more polynomial terms ($x_1^2 x_2, x_1^3 x_2^2, \dots$) the boundary can be any blob. This is Book 3's basis-function trick again — *linear in θ , curved in $\mathbf{x}$* .

STOP AND THINK

A model has $\theta_0 = 2, \theta_1 = -1$ for a single feature x . Where is the boundary, which side is "yes", and what is h at $x = 4$?

OUR ANSWER

$z = 2 - x = 0 \Rightarrow x = 2$. For $x < 2, z > 0$: predict yes. At $x = 4: z = -2, h = 1/(1 + e^2) = 1/(1 + 7.389) = 0.119$ — a 12% chance of yes, predict no.

EXAM TIP

Two easy marks: (1) "classify this point" = compute z , state the sign, then (only if asked for the probability) compute $1/(1 + e^{-z})$; (2) "interpret the coefficient" = odds ratio e^{θ_i} per unit increase, holding other features fixed. Remember e^{-z} for the sigmoid — the most common slip is using e^{+z} .

IN ONE BREATH

$h_{\theta}(\mathbf{x}) = \frac{1}{1+e^{-\theta^T \mathbf{x}}} = P(y = 1 \mid \mathbf{x})$. Predict 1 iff $z = \theta^T \mathbf{x} \geq 0$; the boundary is $z = 0$. $\ln \frac{p}{1-p} = z$, so e^{θ_i} is the odds ratio per unit of x_i . $g' = g(1 - g)$.

CHAPTER 3

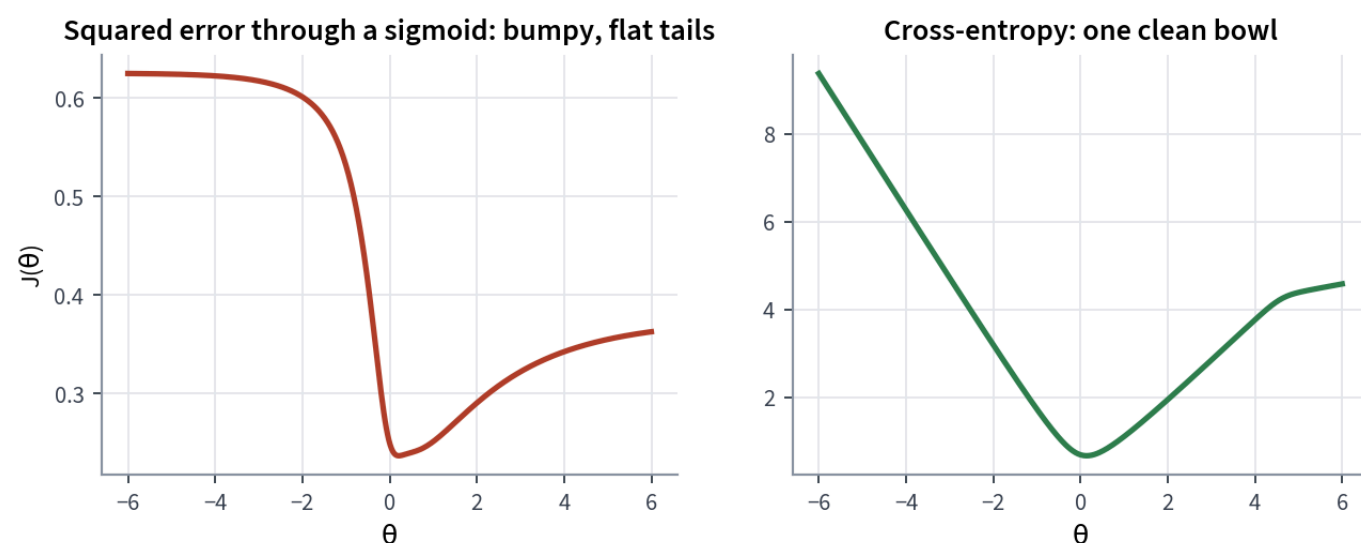
The Cost That Stays a Bowl

Why squared error fails, the $-\log$ cost, cross-entropy, and the gradient that turns out identical to linear regression's.

STORY

Meera tries to reuse Book 3 wholesale: squared error between $h_\theta(\mathbf{x})$ and y , gradient descent. The loss curve wobbles, stalls on a plateau, and two different starting points give two different answers. Arjun: "You pushed a bowl through an S. It is not a bowl any more. You need a cost that was designed for probabilities."

The problem with squared error



The same eight training points, one parameter θ . Left: squared error after the sigmoid — a non-convex curve with a plateau and a spurious dip. Right: cross-entropy — one clean bowl with one bottom.

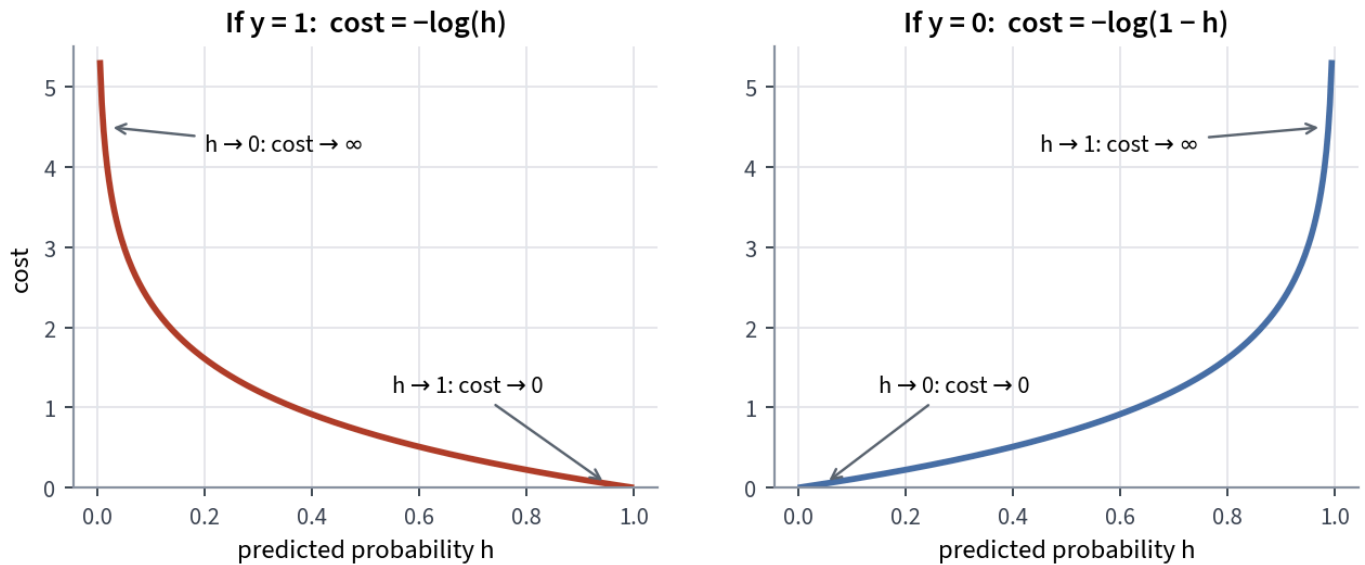
Plugging the sigmoid into $\frac{1}{2n} \sum (h_\theta(\mathbf{x}^{(i)}) - y^{(i)})^2$ produces a cost with many local minima and flat regions where the gradient is nearly zero. Gradient descent may stop in the wrong valley or crawl across a plateau. We need a cost that is **convex** in θ .

The $-\log$ cost, one example at a time

BIG IDEA

$$\text{Cost}(h_\theta(\mathbf{x}), y) = \begin{cases} -\log(h_\theta(\mathbf{x})) & \text{if } y = 1 \\ -\log(1 - h_\theta(\mathbf{x})) & \text{if } y = 0 \end{cases}$$

Punish the model by how much probability it failed to give the true class — on a log scale, so that confident mistakes are punished enormously.



Read the left curve with the slide's job example: the true answer is "offered" ($y = 1$). If the model predicts $h = 1$, cost $= -\log 1 = 0$: perfect. If it predicts $h \rightarrow 0$ ("certainly not offered") the cost $\rightarrow \infty$: we penalise a confidently wrong learner by a very large amount. Exactly what a probability estimate should be charged.

THINK OF IT LIKE

A weather forecaster's bonus. Saying "90% rain" when it rains is a small cost; saying "10% rain" when it rains is a large one; saying "0% rain" and getting soaked is unforgivable — infinite cost. Squared error would charge only $(0 - 1)^2 = 1$ for that last disaster; the log cost charges what it deserves.

One formula for both cases: cross-entropy

Because y is either 0 or 1, we can fold the two cases into one line — when $y = 1$ the second term vanishes, when $y = 0$ the first does:

$$\text{Cost}(h_{\theta}(\mathbf{x}), y) = -y \log h_{\theta}(\mathbf{x}) - (1 - y) \log (1 - h_{\theta}(\mathbf{x})).$$

Average over the m training examples to get the **cross-entropy cost**:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log h_{\theta}(\mathbf{x}^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(\mathbf{x}^{(i)})) \right].$$

This function is convex in θ (it is the negative log-likelihood of a Bernoulli model, and the log of the sigmoid is concave), so gradient descent finds its single minimum.

The gradient — and a small miracle

MATHS, STEP BY STEP — $\partial J / \partial \theta_J$

Work with one example first and write $h = g(z)$, $z = \theta^T \mathbf{x}$. The cost for that example is $c = -y \log h - (1 - y) \log(1 - h)$.

1 Differentiate c with respect to h :

$$\frac{\partial c}{\partial h} = -\frac{y}{h} + \frac{1-y}{1-h}.$$

2 Differentiate h with respect to z — the sigmoid identity from Chapter 2: $\frac{\partial h}{\partial z} = h(1-h)$.**3 Differentiate z with respect to θ_j : $\frac{\partial z}{\partial \theta_j} = x_j$.****4 Chain them:**

$$\frac{\partial c}{\partial \theta_j} = \left(-\frac{y}{h} + \frac{1-y}{1-h} \right) h(1-h) x_j = (-y(1-h) + (1-y)h) x_j.$$

5 Simplify the bracket: $-y + yh + h - yh = h - y$. So

$$\frac{\partial c}{\partial \theta_j} = (h - y) x_j.$$

6 Average over the examples:

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

SPARK

Compare with Book 3: the gradient of linear regression was $\frac{1}{n} \sum (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}$ — **the same expression**. The $h(1-h)$ from the sigmoid's derivative cancelled exactly against the $\frac{1}{h(1-h)}$ hiding in the log cost. The only difference between the two algorithms is what h_{θ} is: $\theta^T \mathbf{x}$ for regression, $g(\theta^T \mathbf{x})$ for classification. That is why the slides show the two update rules side by side, identical except for the red and green boxes.

PLAIN MATHS — GRADIENT DESCENT

Repeat until convergence, all j simultaneously:

$$\theta_j \leftarrow \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

with $h_{\theta}(\mathbf{x}) = \frac{1}{1 + e^{-\theta^T \mathbf{x}}}$.

MATRIX FORM

$\mathbf{h} = g(\mathbf{X}\theta)$ (sigmoid applied element-wise)

$$\nabla J = \frac{1}{m} \mathbf{X}^T (\mathbf{h} - \mathbf{y})$$

$$\theta \leftarrow \theta - \alpha \nabla J$$

Identical to Book 3 with $\mathbf{X}\theta$ replaced by $g(\mathbf{X}\theta)$.

To make a prediction for a new $\mathbf{x}$: output $h_{\theta}(\mathbf{x}) = \frac{1}{1 + e^{-\theta^T \mathbf{x}}}$ and threshold at 0.5. There is **no closed-form solution** for logistic regression — the sigmoid makes the zero-gradient equations non-linear — so gradient descent (or a cousin like Newton's method) is the way.

EXAM TIP

Two derivations are examinable: (1) why squared error is unsuitable (non-convex after the sigmoid; confident mistakes under-penalised) and (2) the gradient of cross-entropy. For (2), write the three chain-rule pieces ($\partial c / \partial h$, $h(1 - h)$, x_j) and the cancellation to $(h - y)x_j$ — that cancellation line is where the marks are.

IN ONE BREATH

Cost per example = $-y \log h - (1 - y) \log(1 - h)$; average = cross-entropy J , which is convex. Its gradient is $\frac{1}{m} \sum (h - y)x_j$, exactly linear regression's formula with h now the sigmoid. Update $\theta_j \leftarrow \theta_j - \alpha \partial J / \partial \theta_j$; no closed form exists.

CHAPTER 4

Fitting by Hand: The Job-Offer Example

One full iteration of gradient descent on the slide's CGPA/IQ data, the sentiment vector, and regularisation with C .

STORY

Before letting scikit-learn near the loan data, Arjun makes Meera do the slide's six-row job-offer table by hand. "Six sigmoids, six errors, three sums, three updates. When you can do this on paper, you will never again be mystified by a training log."

The problem (slide)

i	CGPA x_1	IQ x_2 (scaled)	IQ (raw)	Job offered y
1	5.5	6.7	100	1
2	5	7	105	0
3	8	6	90	1
4	9	7	105	1
5	6	8	120	0
6	7.5	7.3	110	0

Hyper-parameters: learning rate $\alpha = 0.3$, initial weights $(\theta_0, \theta_{\text{CGPA}}, \theta_{\text{IQ}}) = (0.5, 0.5, 0.5)$, regularisation constant 0. The slide uses the *scaled* IQ column (raw IQ $\div 15$); the raw column is shown greyed for reference.

STEP 1 — THE MODEL AND THE UPDATE EQUATIONS

$$z = \boldsymbol{\theta}^T \mathbf{x} = 0.5 + 0.5 \text{ CGPA} + 0.5 \text{ IQ}, \quad h_{\boldsymbol{\theta}}(\mathbf{x}) = \frac{1}{1 + e^{-z}}$$

$$\theta_0 \leftarrow \theta_0 - 0.3 \cdot \frac{1}{6} \sum_{i=1}^6 (h^{(i)} - y^{(i)}) \cdot 1, \quad \theta_{\text{CGPA}} \leftarrow \theta_{\text{CGPA}} - 0.3 \cdot \frac{1}{6} \sum_{i=1}^6 (h^{(i)} - y^{(i)}) x_{\text{CGPA}}^{(i)}, \quad \theta_{\text{IQ}} \leftarrow \theta_{\text{IQ}} - 0.3 \cdot \frac{1}{6} \sum_{i=1}^6 (h^{(i)} - y^{(i)}) x_{\text{IQ}}^{(i)}$$

STEP 2 — PREDICTIONS, ERRORS, AND THE THREE SUMS

i	$z = 0.5 + 0.5x_1 + 0.5x_2$	$h = 1/(1 + e^{-z})$	y	$e = h - y$	$e x_1$	$e x_2$
1	$0.5 + 2.75 + 3.35 = 6.6$	0.9986	1	-0.0014	-0.0075	-0.0091
2	$0.5 + 2.5 + 3.5 = 6.5$	0.9985	0	+0.9985	+4.9925	+6.9895
3	$0.5 + 4 + 3 = 7.5$	0.9994	1	-0.0006	-0.0044	-0.0033
4	$0.5 + 4.5 + 3.5 = 8.5$	0.9998	1	-0.0002	-0.0018	-0.0014
5	$0.5 + 3 + 4 = 7.5$	0.9994	0	+0.9994	+5.9967	+7.9956
6	$0.5 + 3.75 + 3.65 = 7.9$	0.9996	0	+0.9996	+7.4972	+7.2972
Sums				2.9955	18.4727	22.2685

Notice what happened: with all weights at +0.5, every z is between 6.5 and 8.5, so the sigmoid says " ≈ 1 " for everybody. The three "yes" rows have almost zero error; the three "no" rows each contribute an error of almost exactly 1. The sums are therefore ≈ 3 , $\approx 5 + 6 + 7.5 = 18.5$ and $\approx 7 + 8 + 7.3 = 22.3$ — a useful sanity check you can do without a calculator.

STEP 3 — THE UPDATES

- 1 $\theta_0 = 0.5 - 0.3 \cdot \frac{2.9955}{6} = 0.5 - 0.3(0.4992) = 0.5 - 0.1498 = \mathbf{0.350}$
- 2 $\theta_{\text{CGPA}} = 0.5 - 0.3 \cdot \frac{18.4727}{6} = 0.5 - 0.3(3.0788) = 0.5 - 0.9236 = \mathbf{-0.424}$
- 3 $\theta_{\text{IQ}} = 0.5 - 0.3 \cdot \frac{22.2685}{6} = 0.5 - 0.3(3.7114) = 0.5 - 1.1134 = \mathbf{-0.613}$

The slide reports these rounded: $\theta_0 \approx 0.4$, $\theta_{\text{CGPA}} \approx -0.4$, $\theta_{\text{IQ}} \approx -0.6$. Cross-entropy falls from $J = 3.65$ to $J = 3.43$.

CAREFUL — WHAT A SECOND ITERATION SHOWS

With the new weights every z is now strongly negative (≈ -5 to -8), all six h are ≈ 0 , and the three "yes" rows now carry errors of ≈ -1 . The second update pushes the weights back up to about (0.50, 0.70, 0.37) and J rises to 3.78. The learner is **oscillating**: with unscaled inputs (z swings by ± 7) a learning rate of 0.3 is too large. In an exam, report the numbers, note that the loss went up, and prescribe a smaller α or standardised features — exactly Book 3's lesson.

Prediction and interpretation (slide)

Suppose after training the model is $z = 0.4 + 0.3 \text{ CGPA} - 0.45 \text{ IQ}$. For a candidate with CGPA 5 and IQ 6: $z = -0.8$, $h = 0.31$, predicted **No**. Coefficient reading: $e^{0.3} = 1.35$ — each extra CGPA point multiplies the odds of an offer by 1.35 (Chapter 2).

Another worked application: sentiment with engineered features (slide)

A movie review is turned into six hand-made features: x_1 = number of positive-lexicon words (3), x_2 = negative-lexicon words (2), $x_3 = 1$ if "no" occurs (1), x_4 = count of 1st/2nd-person pronouns (3), $x_5 = 1$ if "!" occurs (0), $x_6 = \ln(\text{word count}) = \ln 66 = 4.19$. Learned weights $\mathbf{w} = [2.5, -5.0, -1.2, 0.5, 2.0, 0.7]$, bias $b = 0.1$.

WORKED EXAMPLE

$$z = \mathbf{w} \cdot \mathbf{x} + b = 2.5(3) - 5.0(2) - 1.2(1) + 0.5(3) + 2.0(0) + 0.7(4.19) + 0.1 = 7.5 - 10 - 1.2 + 1.5 + 0 + 2.933 + 0.1 = 0.833$$

$$P(+ | \mathbf{x}) = \sigma(0.833) = \frac{1}{1 + e^{-0.833}} = \frac{1}{1 + 0.4347} = 0.70, \quad P(- | \mathbf{x}) = 1 - 0.70 = 0.30.$$

The review is classified positive with 70% confidence. The weights are readable: positive words push up (+2.5 each), negative words push down hard (−5 each), an exclamation mark helps (+2).

Regularisation for logistic regression

Overfitting works exactly as in Book 3 and so does the cure: add a penalty on the weights to the cross-entropy cost. The slide writes it in the L1 form,

$$\text{Loss}(S) = \sum_i \text{Loss}(\hat{y}_i, y_i) + \alpha \sum_j |\theta_j|,$$

and the L2 form $\frac{\lambda}{2m} \sum_j \theta_j^2$ is equally common; the gradient update gains the same extra term as ridge or lasso ($\theta_j(1 - \alpha\lambda/m)$ shrink, or $-\alpha\lambda \text{sign}(\theta_j)$), and θ_0 is left alone. Everything about choosing λ on a validation set carries over.

CAREFUL — C , NOT α (SLIDE NOTE)

In scikit-learn's `LogisticRegression` the regularisation strength is not `alpha` but its **inverse**, C : $C = 1/\lambda$. **Higher C = less regularisation** (a more flexible model); lower C = stronger regularisation. Students routinely get this backwards.

EXAM TIP — PROBLEM TYPE "FIT A LOGISTIC MODEL"

Layout that scores: (1) write z and h with the initial weights; (2) a table with $z, h, y, e = h - y, e x_1, e x_2$; (3) the three sums; (4) the three updates with α/m shown; (5) if regularised, the extra term per weight (not on θ_0); (6) predict for the new point using the *updated* weights and state the class from the sign of z . Show e^{-z} values to two or three decimals.

IN ONE BREATH

Tabulate $z, h, e = h - y, e x_j$; sum; update $\theta_j \leftarrow \theta_j - \frac{\alpha}{m} \sum e x_j$. Predict by the sign of z ; interpret by e^{θ_j} . Regularise like ridge/lasso; in scikit-learn $C = 1/\lambda$, bigger C means weaker regularisation.

CHAPTER 5

Is the Classifier Any Good? The Confusion Matrix

Accuracy and why it lies, precision, recall, specificity, F-score — with the skewed-class tables worked out.

STORY

Meera's loan model reports 99% accuracy. Priya is thrilled until Arjun asks how many defaulters it caught. "None. It says 'will repay' to everyone, and 99% of people repay." One number had hidden a useless model. Meera needs four numbers, not one.

The confusion matrix

		PREDICTED CLASS		
		Positive (Yes)	Negative (No)	
ACTUAL CLASS	Positive (Yes)	TP true positive — hit	FN false negative — miss	row = actual positives = TP + FN
	Negative (No)	FP false positive — false alarm	TN true negative — correct rejection	row = actual negatives = FP + TN

Entry $CM_{i,j}$ counts tuples of actual class i labelled by the classifier as class j . The diagonal is right; the off-diagonal is wrong. Rows sum to the actual class sizes.

TP Predicted positive and actually positive. A defaulter correctly flagged.

TN Predicted negative and actually negative. A good borrower correctly approved.

FP Predicted positive but actually negative. A good borrower wrongly refused (a false alarm; "Type I error").

FN Predicted negative but actually positive. A defaulter wrongly approved (a miss; "Type II error").

THINK OF IT LIKE

Airport security. TP: a threat caught. TN: a harmless passenger waved through. FP: a harmless passenger pulled aside — annoying, costs time. FN: a threat waved through — the disaster. The two mistakes are not equal, and no single number should pretend they are.

The metrics

DEFINITIONS

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Error rate} = 1 - \text{Accuracy}$$

$$\frac{TP}{TP + FP} \quad \text{Precision (positive predictive value) = "of those I called positive, how many really were?"}$$

$$\text{Recall} = \text{Sensitivity} = \text{TPR} = \frac{TP}{TP + FN} \quad \text{"of the real positives, how many did I catch?"}$$

$$\text{Specificity} = \text{TNR} = \frac{TN}{TN + FP}, \quad \text{FPR} = \frac{FP}{FP + TN} = 1 - \text{Specificity}, \quad \text{FNR} = \frac{FN}{FN + TP} = 1 - \text{Sensitivity}$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (\text{the harmonic mean — high only when both are high})$$

Precision is about the *column* of predicted positives; recall is about the *row* of actual positives. The F_1 score combines them into a single number and is the standard metric when classes are imbalanced. (The slide also lists "Power = sensitivity = $1 - \beta$ " from hypothesis-testing language: α is the FPR, β the FNR.)

Why accuracy lies: the skew problem

WORKED EXAMPLE (SLIDE) — TWO MODELS, 1000 CASES, 10 POSITIVES

	Pred Yes	Pred No
Actual Yes	0 (TP)	10 (FN)
Actual No	0 (FP)	990 (TN)

Model A: says "No" to everyone. Accuracy = $990/1000 = 99\%$. Recall = $0/10 = 0$. Useless.

	Pred Yes	Pred No
Actual Yes	10 (TP)	0 (FN)
Actual No	500 (FP)	490 (TN)

Model B: catches every positive, with many false alarms. Accuracy = $500/1000 = 50\%$. Recall = 1. Precision = $10/510 = 2\%$.

Accuracy says A is far better; anyone hunting frauds, intrusions or defects wants B (and then works on its precision). Accuracy is most effective only when the class distribution is relatively balanced. Detecting the rare class is usually what matters.

Which classifier is better? Three skew levels (slides)

Three classifiers T1, T2, T3 are evaluated on the same 100 positives, with the number of negatives growing from 100 to 1,000 to 10,000. Each metric is computed with the formulas above; every number below can be recomputed in a line.

Skew	Model	TP	FN	FP	TN	Precision	Recall (TPR)	FPR	TPR/FPR	F_1	Accuracy
Low (100 neg)	T1	50	50	1	99	$50/51 = 0.98$	0.50	0.01	50	0.66	0.745
	T2	99	1	10	90	$99/109 = 0.91$	0.99	0.10	9.9	0.95	0.945
	T3	99	1	1	99	0.99	0.99	0.01	99	0.99	0.99
Medium (1,000 neg)	T1	50	50	10	990	$50/60 = 0.83$	0.50	0.01	50	0.625	0.946
	T2	99	1	100	900	$99/199 = 0.50$	0.99	0.10	9.9	0.66	0.908
	T3	99	1	10	990	$99/109 = 0.91$	0.99	0.01	99	0.95	0.99
High (10,000 neg)	T1	50	50	100	9900	$50/150 = 0.33$	0.50	0.01	50	0.40	0.985
	T2	99	1	1000	9000	$99/1099 = 0.09$	0.99	0.10	9.9	0.165	0.901
	T3	99	1	100	9900	$99/199 = 0.50$	0.99	0.01	99	0.66	0.99

READING THE TABLE

- **Recall, FPR and TPR/FPR never change with skew** — they are computed within a row of the confusion matrix, so the number of negatives cannot affect recall and the number of positives cannot affect FPR. T3 has the best of both (TPR 0.99, FPR 0.01) at every skew level.
- **Precision and F_1 collapse as skew grows.** T2's precision goes $0.91 \rightarrow 0.50 \rightarrow 0.09$: the same 10% false-alarm rate, applied to ten thousand negatives, produces a thousand false alarms that swamp the 99 true hits. Precision depends on the class ratio; recall and FPR do not.
- **Accuracy is nearly blind.** In the high-skew case T1 (which misses half the positives) has 98.5% accuracy — higher than T2, which catches 99% of them.

Verdict: T3 is best at every skew; between T1 and T2 the answer depends on whether misses (FN) or false alarms (FP) cost more.

Mistakes have different costs

	False positive rate	False negative rate
Model 1	41%	3%
Model 2	5%	25%

Which should you use? It depends on the application. **Disease screening** wants a **low FN rate** — missing a cancer is far worse than a needless follow-up test — so Model 1. **Spam filtering** wants a **low FP rate** — deleting a real email is worse than letting a spam through — so Model 2. The same application may need **conservative and aggressive settings** at different times; Chapter 6 shows how one model can offer both by moving its threshold.

Worked example — the medical table (slide practice)

CANCER = YES IS THE POSITIVE CLASS

Actual \ Predicted	yes	no	Total
yes	90	210	300
no	140	9560	9700
Total	230	9770	10,000

- 1 $TP = 90, FN = 210, FP = 140, TN = 9560$.
- 2 Accuracy = $(90 + 9560)/10,000 = 0.965$; error rate = $350/10,000 = 0.035$.
- 3 Precision = $90/(90 + 140) = 90/230 = 0.391$.
- 4 Recall = $90/(90 + 210) = 90/300 = 0.300$.
- 5 $F_1 = 2(0.391)(0.300)/(0.391 + 0.300) = 0.2346/0.691 = 0.340$.

Interpretation: 96.5% accuracy looks superb, but the classifier catches only 30% of cancers and is wrong about 61% of the patients it flags. For a screening test this is unacceptable — recall must go up (accept more false positives). Precision would matter more if the flag triggered an expensive or risky intervention.

EXAM TIP

Always label which class is "positive" first. Then write TP, FN, FP, TN as four separate lines before any formula. Compute accuracy, precision, recall, specificity, F_1 ; end with one sentence on what the numbers *mean* for that application (which mistake is expensive?). "Brainstorm scenarios where precision is preferred over recall" — precision: spam filter, recommendation, any costly action on a positive; recall: disease screening, fraud, safety inspection, any costly miss.

IN ONE BREATH

Four counts: TP, FN, FP, TN. Accuracy = $(TP + TN)/\text{all}$ — misleading under skew. Precision = $TP/(TP + FP)$, recall = $TP/(TP + FN)$, FPR = $FP/(FP + TN)$, F_1 = harmonic mean of precision and recall. Recall and FPR ignore skew; precision does not. Choose the metric by the cost of each mistake.

CHAPTER 6

ROC Curves and AUC

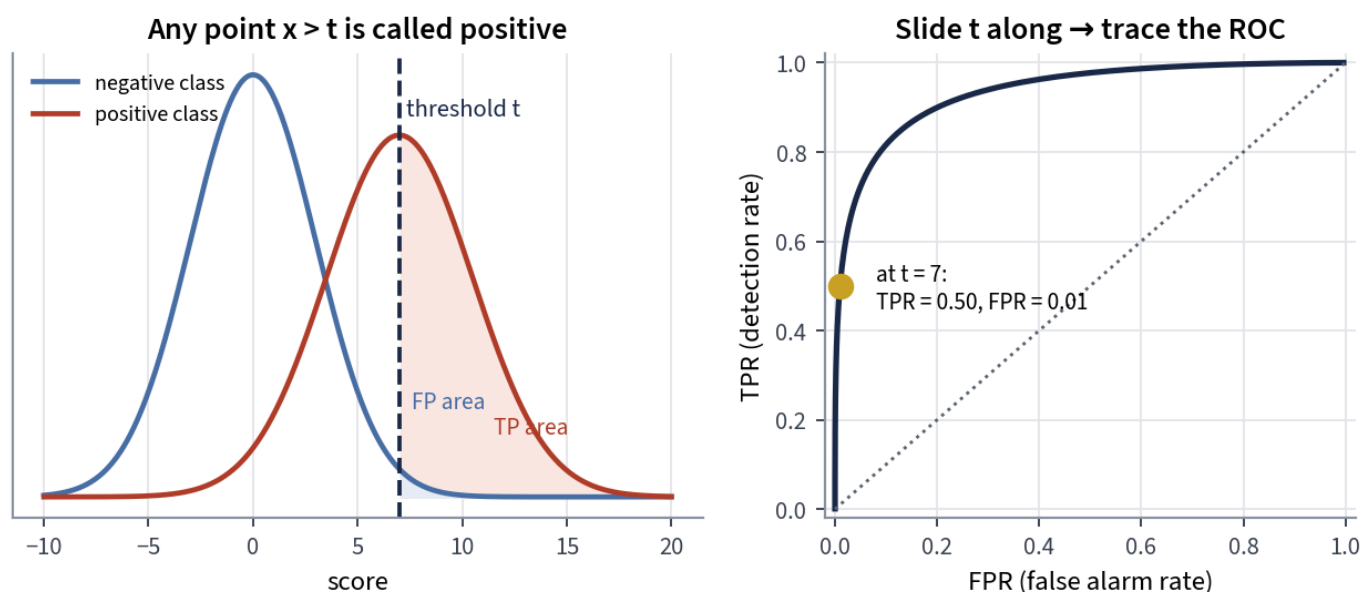
One classifier, every threshold: how to build an ROC curve from scores, compute the area, and compare models.

STORY

Priya wants both: a strict setting for big loans and a lenient one for small loans. "Do I need two models?" asks Meera. "No," says Arjun. "One model, one probability, two thresholds. Plot how the model behaves at *every* threshold and you have its whole personality on one page." That page is the ROC curve.

The threshold is a dial

Logistic regression outputs a score $h_\theta(\mathbf{x}) \in (0, 1)$; we have been cutting at 0.5. Any other cut-off t gives a different classifier: raise t and you call fewer things positive (fewer FP, more FN); lower it and the reverse. Each t has its own (FPR, TPR) pair.



Left: scores of the two classes overlap; every point with score $> t$ is called positive. The red area beyond t is the TPR, the blue area beyond t is the FPR. At $t = 7$ (slide): $\text{TPR} = 0.5$, $\text{FNR} = 0.5$, $\text{FPR} \approx 0.01$, $\text{TNR} \approx 0.99$. Right: slide t from $+\infty$ to $-\infty$ and the pair (FPR, TPR) traces the ROC curve.

BIG IDEA — THE ROC CURVE

The **Receiver Operating Characteristic** curve plots TPR (detection rate, y -axis) against FPR (false-alarm rate, x -axis) for every threshold. It was developed in the 1950s for radar signal detection; the name stuck. Landmarks:

- $(0, 0)$: declare everything negative — no false alarms, no detections. $t = 1$.
- $(1, 1)$: declare everything positive — every detection, every false alarm. $t = 0$.
- $(0, 1)$: the ideal — every positive caught, no false alarms.
- The **diagonal** $TPR = FPR$: random guessing. **Below** the diagonal: worse than random — the predictions are the opposite of the truth (flip them!).

CAREFUL

The slide writes the ideal point as $(TPR, FPR) = (1, 0)$ and the corners as $(0, 0)$ and $(1, 1)$; on the plot, with FPR on the x -axis, the ideal is the *top-left* corner. Whichever order you write the pair, say which axis is which.

How to construct an ROC curve from scores

RECIPE (SLIDE)

1. Use a classifier that gives a continuous score per instance — the higher, the more likely positive.
2. Sort the instances by decreasing score.
3. Put the threshold at each distinct score value in turn ("call positive if score $\geq t$ ").
4. At each threshold count TP, FP, TN, FN; compute $TPR = TP / (TP + FN)$ and $FPR = FP / (FP + TN)$.
5. Plot the points, joined by steps: a positive instance moves the curve *up* by $1/P$, a negative moves it *right* by $1/N$.

WORKED EXAMPLE — TEN TUPLES (SLIDE)

Five positives ($P = 5$), five negatives ($N = 5$), sorted by the classifier's probability:

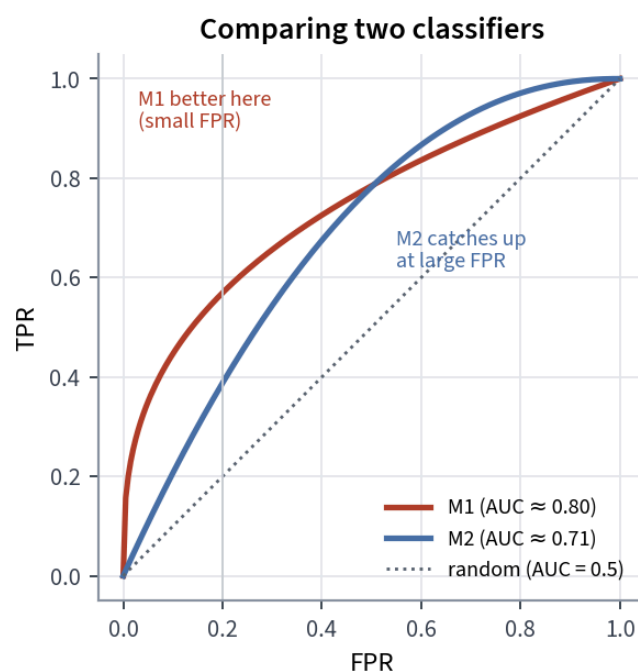
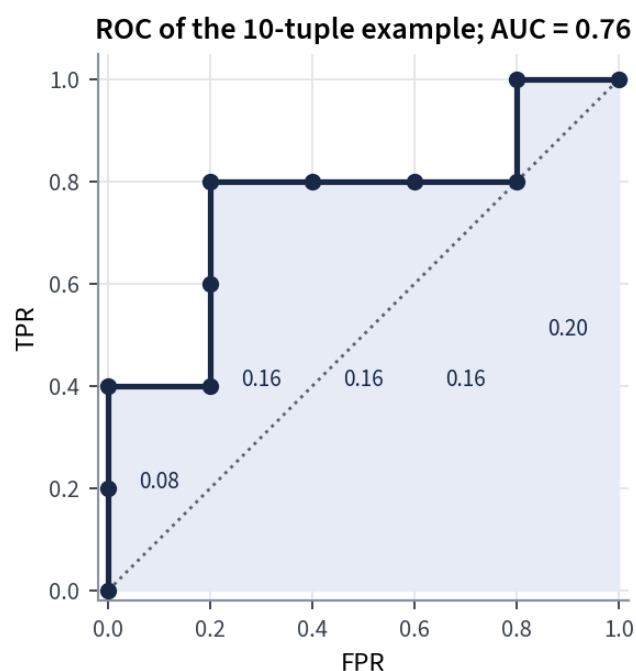
Tuple	Class	Prob.	threshold $t \geq$	TP	FP	TN	FN	TPR	FPR
1	P	0.90	0.90	1	0	5	4	0.2	0
2	P	0.80	0.80	2	0	5	3	0.4	0
3	N	0.70	0.70	2	1	4	3	0.4	0.2
4	P	0.60	0.60	3	1	4	2	0.6	0.2
5	P	0.55	0.55	4	1	4	1	0.8	0.2
6	N	0.54	0.54	4	2	3	1	0.8	0.4
7	N	0.53	0.53	4	3	2	1	0.8	0.6
8	N	0.51	0.51	4	4	1	1	0.8	0.8
9	P	0.50	0.50	5	4	1	0	1.0	0.8
10	N	0.40	0.40	5	5	0	0	1.0	1.0

Read row 4: with $t = 0.60$ the top four tuples are called positive; three of them are truly P ($TP = 3$), one is N ($FP = 1$); the remaining P's are missed ($FN = 2$) and the remaining N's are correctly rejected ($TN = 4$). $TPR = 3/5 = 0.6$, $FPR = 1/5 = 0.2$. Each P moves the curve up by 0.2; each N moves it right by 0.2.

CAREFUL — TIED SCORES

In the other slide example (instances scored 0.95, 0.93, 0.87, 0.85, 0.85, 0.85, 0.76, ...), three instances share the score 0.85. A threshold " ≥ 0.85 " must admit **all three at once** — you cannot separate tied instances — so the curve jumps diagonally by (two negatives, one positive) in a single step. Always group ties.

AUC: the area under the curve



Left: the step curve from the table above, with the area split into rectangles under each horizontal run. Right: two classifiers cross — neither dominates; AUC summarises each with one number.

MATHS, STEP BY STEP — AUC BY THE TRAPEZOIDAL RULE

List the ROC points in order of increasing FPR: $(x_0, y_0), (x_1, y_1), \dots$. The area under the polyline is the sum of trapezoids:

$$\text{AUC} = \sum_i (x_{i+1} - x_i) \cdot \frac{y_i + y_{i+1}}{2}.$$

Vertical moves ($x_{i+1} = x_i$) contribute nothing; for a step curve every horizontal run is a rectangle of width Δx and height the current TPR.

WORKED EXAMPLE — AUC OF THE TEN-TUPLE CURVE

Horizontal runs (from the table): FPR $0 \rightarrow 0.2$ at TPR 0.4; $0.2 \rightarrow 0.4$, $0.4 \rightarrow 0.6$, $0.6 \rightarrow 0.8$ all at TPR 0.8; $0.8 \rightarrow 1.0$ at TPR 1.0.

$$\text{AUC} = 0.2(0.4) + 0.2(0.8) + 0.2(0.8) + 0.2(0.8) + 0.2(1.0) = 0.08 + 0.16 + 0.16 + 0.16 + 0.20 = \mathbf{0.76}.$$

(The slide's separate AUC example lists points $(1.0, 0.8), (0.8, 0.8), (0.8, 0.6), \dots$ in decreasing-FPR order and gets $0.16 + 0.12 + 0.12 + 0.12 + 0.08 = 0.60$ by the same rule — the average of the two TPRs times the width for each horizontal step.)

AUC = 1 Perfect separability: some threshold catches all positives with no false alarms.

AUC = 0.5 Random guessing — the diagonal.

AUC < 0.5 Systematically wrong; flip the labels.

A nice interpretation: **AUC is the probability that a randomly chosen positive gets a higher score than a randomly chosen negative.** That is why it measures "degree of separability" and does not depend on any one threshold or on class skew.

Using ROC curves

- **Threshold selection.** Pick the point on the curve that matches the costs: a screening test wants high TPR (accept larger FPR); a spam filter wants tiny FPR (accept lower TPR). Priya's two settings are two points on one curve.
- **Performance assessment.** How far above the diagonal is the curve? AUC.
- **Classifier comparison.** In the slide's picture M1 is better for small FPR and M2 for large FPR — no model consistently outperforms the other. If the operating region is known, compare there; otherwise compare AUC (larger is better).

STOP AND THINK

Model A has AUC 0.90 but at FPR = 0.05 its TPR is 0.40. Model B has AUC 0.80 but at FPR = 0.05 its TPR is 0.70. You are building a spam filter. Which do you deploy?

OUR ANSWER

B. A spam filter must operate at very low FPR (losing real mail is expensive), and in that region B catches far more spam. AUC averages over all thresholds, including ones you will never use; when you know your operating point, compare there.

EXAM TIP

ROC construction is procedural and generous with marks: a table with one row per distinct score, columns TP / FP / TN / FN / TPR / FPR, then the sketch. For AUC, list the horizontal segments and their rectangle (or trapezoid) areas explicitly, then add. State P and N up front so every TPR is "TP over P ".

IN ONE BREATH

Slide the threshold; each position gives (FPR, TPR); the ROC curve is all of them. Build it by sorting scores and counting. $AUC = \text{sum of trapezoids} = \text{probability a positive outscored a negative}$; 1 perfect, 0.5 random. Use the curve to pick a threshold for your costs and to compare models.

CHAPTER 7

Rare Classes and Many Classes

What to do when the class you care about is rare, and how a two-class model handles three or more classes.

STORY

Only 1% of Sahi Deal's borrowers default; the model learnt "everyone repays". Meanwhile Marketing wants listings sorted into *hatchback* / *sedan* / *SUV* — three classes, and logistic regression only knows yes/no. Two last tools.

Class imbalance

The main class of interest is often **rare**: fraud among transactions, cancer among test results, defaults among borrowers. Then the training data is dominated by the negative class, the learner is rewarded for ignoring the positives, and **accuracy becomes meaningless** (Chapter 5). Three remedies from the slides:

1. Generate synthetic samples

Oversample the minority class — not just by copying rows, but by creating new points between a minority example and its nearest minority neighbours. **SMOTE** (Synthetic Minority Oversampling Technique) does exactly this (available in the `imblearn` library). Undersampling the majority class is the mirror option.

2. Change the metric

Optimise and report **recall, precision, F_1 , or ROC/AUC** instead of accuracy. These see the minority class; accuracy does not.

3. Try different algorithms

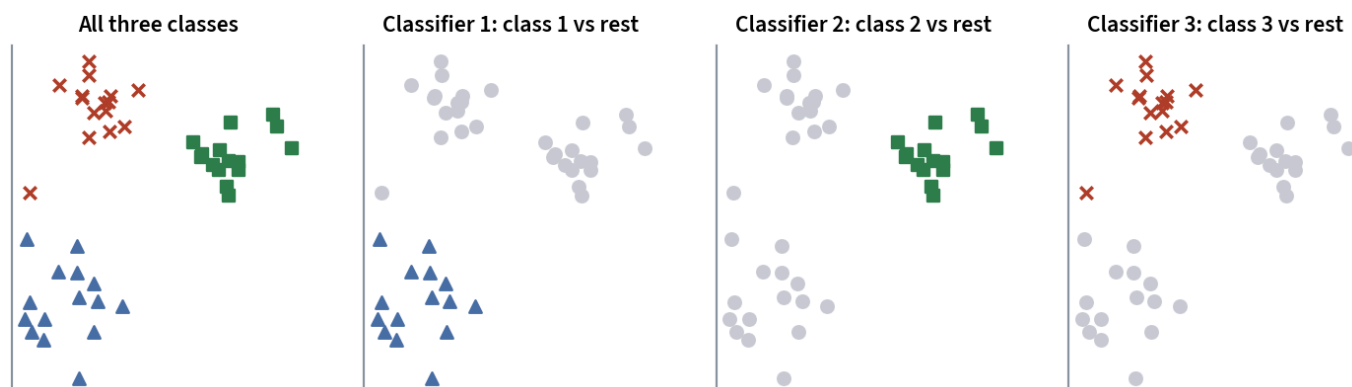
Support vector machines and tree-based methods can cope better with imbalance (and accept class weights). These come after the mid-term.

CAREFUL

Resample the **training set only**. The validation and test sets must keep the real class proportions, or your metrics will describe a world that does not exist.

Multi-class classification

Logistic regression is a binary tool: one sigmoid, one boundary. For K classes ({yes, no, maybe}; {hatchback, sedan, SUV}) there are two standard constructions.



One-vs-all: three binary problems, each treating one class as positive and everything else as negative.

One-vs-all (one-vs-rest, OvA)

Train K classifiers. Classifier i learns $h_{\theta}^{(i)}(\mathbf{x}) = P(y = i \mid \mathbf{x}; \theta)$, treating class i as "1" and all other classes as "0". For a new $\mathbf{x}$, run all K and **predict the class whose classifier is most confident**: $\hat{y} = \arg \max_i h_{\theta}^{(i)}(\mathbf{x})$. K models, each trained on all the data.

One-vs-one (OvO)

Train one classifier for **every pair** of classes — class 1 vs 2, 1 vs 3, 2 vs 3 — that is $\frac{K(K-1)}{2}$ classifiers, each trained only on the data of its two classes. For a new $\mathbf{x}$ every classifier votes; the class with the most votes wins. More models, but each is small and fast to train.

WORKED EXAMPLE — HOW MANY CLASSIFIERS?

Ten digit classes. OvA: 10 classifiers. OvO: $10 \times 9 / 2 = 45$ classifiers. For $K = 3$: OvA 3, OvO 3 — the same count; the two strategies differ above $K = 3$.

Scikit-learn's note (slide): when you hand a binary algorithm a multi-class task it automatically runs OvA — except for SVM classifiers, where it uses OvO, because SVM training scales badly with data size and many small problems are cheaper than a few big ones. (A third route, **softmax** regression, generalises the sigmoid to K outputs directly; it is not on these slides.)

The summary slide, annotated

Piece	Formula	In words
Model	$h_{\theta}(\mathbf{x}) = P(Y = 1 \mid X_1, \dots, X_n) = \frac{1}{1 + e^{-\theta^T \mathbf{x}}}$	Sigmoid of a linear score; a probability.
Cost	$J(\theta) = \frac{1}{m} \sum_i \text{Cost}(h_{\theta}(\mathbf{x}^{(i)}), y^{(i)})$, $\text{Cost} = -\log h$ if $y = 1$, $-\log(1 - h)$ if $y = 0$	Cross-entropy; convex.
Learning	$\theta_j \leftarrow \theta_j - \alpha \frac{1}{m} \sum_i (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}$	Gradient descent, same form as linear regression.
Inference	$\hat{Y} = h_{\theta}(\mathbf{x}^{\text{test}}) = \frac{1}{1 + e^{-\theta^T \mathbf{x}^{\text{test}}}}$, predict 1 if ≥ 0.5	Equivalently, predict 1 iff $\theta^T \mathbf{x} \geq 0$.
Logit	$\text{logit}(p) = \ln \frac{p}{1 - p} = \theta^T \mathbf{x}$	The inverse of the sigmoid; the log-odds are linear.

IN ONE BREATH

Rare class: resample (SMOTE) the training set, judge by recall/precision/ F_1 /AUC, consider SVMs or trees.
 Many classes: one-vs-all (K models, pick the most confident) or one-vs-one ($K(K - 1)/2$ models, majority vote).

CHAPTER 8

Formula Sheet, Self-Check, and What Comes Next

Everything from Sessions 5–6 on one page, twelve questions with answers, and the bridge to decision trees.

The formula sheet

Logistic regression

$z = \boldsymbol{\theta}^\top \mathbf{x}$; $h_{\boldsymbol{\theta}}(\mathbf{x}) = g(z) = \frac{1}{1 + e^{-z}} = P(y = 1 \mid \mathbf{x})$; $g(0) = 0.5$; $1 - g(z) = g(-z)$; $g'(z) = g(z)(1 - g(z))$; $\ln \frac{p}{1-p} = z$; odds ratio per unit of x_j : e^{θ_j} .

Predict 1 iff $z \geq 0$. Boundary: $\boldsymbol{\theta}^\top \mathbf{x} = 0$ (add $x_1^2, x_2^2, \dots$ for curved boundaries).

Cost and learning

$J(\boldsymbol{\theta}) = -\frac{1}{m} \sum_i [y^{(i)} \log h^{(i)} + (1 - y^{(i)}) \log(1 - h^{(i)})]$; $\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_i (h^{(i)} - y^{(i)}) x_j^{(i)}$; $\theta_j \leftarrow \theta_j - \alpha \frac{\partial J}{\partial \theta_j}$; matrix: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \frac{\alpha}{m} \mathbf{X}^\top (g(\mathbf{X}\boldsymbol{\theta}) - \mathbf{y})$. Regularised: add $\frac{\lambda}{2m} \sum_{j \geq 1} \theta_j^2$ (L2) or $\lambda \sum_{j \geq 1} |\theta_j|$ (L1); scikit-learn $C = 1/\lambda$.

Evaluation

$\text{Acc} = \frac{TP+TN}{TP+TN+FP+FN}$; $\text{Prec} = \frac{TP}{TP+FP}$; $\text{Rec} = \text{TPR} = \frac{TP}{TP+FN}$; $\text{Spec} = \text{TNR} = \frac{TN}{TN+FP}$; $\text{FPR} = \frac{FP}{FP+TN}$; $\text{FNR} = \frac{FN}{FN+TP}$; $F_1 = \frac{2PR}{P+R}$.

ROC: TPR vs FPR over thresholds; (0, 0) all-negative, (1, 1) all-positive, top-left ideal, diagonal random.

$\text{AUC} = \sum_i (x_{i+1} - x_i) \frac{y_i + y_{i+1}}{2}$; 1 perfect, 0.5 random.

Multi-class

One-vs-all: K classifiers, $\hat{y} = \arg \max_i h_{\boldsymbol{\theta}}^{(i)}(\mathbf{x})$. One-vs-one: $K(K-1)/2$ classifiers, majority vote.

Twelve questions (cover the right column)

#	Question	Answer
1	Two reasons least squares is unsuitable for classification.	Outputs leave $[0, 1]$; the fitted line is dragged by easy/outlying points so the 0.5 cut-off moves.
2	Why does logistic regression predict 1 exactly when $\theta^T \mathbf{x} \geq 0$?	$g(z) \geq 0.5 \iff z \geq 0$ since $g(0) = 0.5$ and g is increasing.
3	What is the logit and how does it relate to the sigmoid?	$\text{logit}(p) = \ln \frac{p}{1-p}$; it is the inverse of the sigmoid; logistic regression makes the logit linear in $\mathbf{x}$.
4	Coefficient $\theta_{\text{CGPA}} = 0.3$. Interpret it.	Each unit increase in CGPA multiplies the odds of $y = 1$ by $e^{0.3} = 1.35$, other features fixed.
5	Why not squared error as the cost?	Through the sigmoid it is non-convex (local minima, plateaus) and under-penalises confident mistakes.
6	Write the per-example cross-entropy cost and its value when $y = 1, h = 0.01$.	$-y \log h - (1 - y) \log(1 - h)$; $-\log 0.01 = 4.6$ — large.
7	The gradient of J for logistic regression?	$\frac{1}{m} \sum_i (h_\theta(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}$ — identical in form to linear regression.
8	In scikit-learn, does a larger C mean more or less regularisation?	Less: $C = 1/\lambda$.
9	$TP = 90, FN = 210, FP = 140, TN = 9560$: precision, recall, F_1 ?	0.391, 0.300, 0.340; accuracy 0.965 is misleading.
10	Which of precision / recall / FPR change when negatives are multiplied by 10?	Only precision (and accuracy, F_1). Recall and FPR are row-based and unaffected.
11	What do the points $(0, 0)$, $(1, 1)$ and the diagonal mean on an ROC plot?	All-negative classifier; all-positive classifier; random guessing (AUC 0.5).
12	How many classifiers do OvA and OvO train for $K = 5$?	OvA 5; OvO $5 \cdot 4/2 = 10$.

SPARK

Book 3 and Book 4 are one algorithm. Both say: score $= \theta^T \mathbf{x}$; measure the gap between prediction and truth; move θ against the average of (gap $\times$ feature). The only difference is a single function — the identity for regression, the sigmoid for classification — and a cost chosen so that the gradient comes out the same. Once you see that, "linear models" is one chapter of your mind, not two.

STORY — TO BE CONTINUED

The loan model works: AUC 0.87, threshold tuned for Priya's two products. But the compliance officer has a different complaint: "I cannot explain to a rejected customer why a sigmoid said 0.42." Meera needs a model whose reasons a human can read — a sequence of plain questions. That is Book 5: decision trees.